Closed Book Closed Notes No Electronic Devices [no smartwatches/digital watches].

- No Clarifications given in Exam – make Assumptions if you think problem is incompletely/under specified.
- Please use **blue/black pen** to write your answers [**Pencil is not acceptable**].
- Please write your name OR entry number in each page — if that is absent, grading of that page is **NOT** done.
- Please refrain from taking a bathroom break — marks of students who take a bathroom break will be modulated by the marks of a special Viva for these students [*essentially all answers have to defended orally*].

Section 1. MCQ

1. [1] Arrange the following in increasing access time:
   (a) L1 cache, L2 cache, DRAM, Disk
   (b) Disk, DRAM, L2 cache, L1 cache
   (c) L2 cache, L1 cache, DRAM, Disk
   (d) DRAM, L1 cache, L2 cache, Disk
2. [1] GPUs achieve high throughput by:
   (a) Increasing clock speeds
   (b) Running thousands of lightweight threads in parallel
   (c) Having larger caches than CPUs
   (d) Eliminating control logic
3. [1]Which of the following best describes key differences between AVX-256 and AVX-512 instruction sets?
   A)
   (a) AVX-512 supports 512-bit wide vector registers, offers more registers, and includes additional mask registers for predication compared to AVX-256's 256-bit wide vectors.
   (b) AVX-256 is newer and more powerful than AVX-512, with improved support for integer SIMD but no floating-point operations.
   (c) Both AVX-256 and AVX-512 operate on the same register width and only differ in their instruction encoding.
   (d) AVX-512 eliminates all legacy SSE instructions and cannot coexist with them in modern processors.

Set D

4. [1] A superscalar processor differs from a scalar processor because it can:

   (a) Only execute vector instructions
   (b) Issue multiple instructions per cycle
   (c) Execute one instruction per cycle
   (d) Use no pipelining

5. [1] Which factor most often limits the speedup of superscalar processors?

   (a) Clock frequency
   (b) Instruction dependencies and branch mispredictions
   (c) Power supply voltage
   (d) Virtual memory paging

6. [1] Which system software combines multiple object files and libraries into one executable?

   (a) Compiler
   (b) Assembler
   (c) Linker
   (d) Debugger

7. [1] Which GPU execution model corresponds to "Single Instruction Multiple Threads" commonly used in modern GPUs?

   (a) SIMD (incorrect)
   (b) MIMD
   (c) SIMT (correct)
   (d) SISD

8. [1] Which of these best describes **out-of-order execution**?

   (a) Executing instructions in the same order they appear regardless of dependencies.
   (b) Reordering instruction execution to increase resource utilization while respecting data dependencies.
   (c) Execution restricted to in-order only but with speculative loads.
   (d) Only used in GPUs, never in CPUs.

9. [1] Superscalar execution aims to exploit:

   (a) Thread-level parallelism
   (b) Instruction-level parallelism
   (c) Data-level parallelism
   (d) Process-level parallelism

10. [1] SIMT, as used in GPUs, differs from SIMD because:

(a) Each thread can branch independently

(b) Only one instruction can run at a time

(c) It uses MIMD at the hardware level

(d) It eliminates vectorization

11. [1] A branch predictor is used to reduce:

(a) Cache misses

(b) Pipeline stalls

(c) Disk access time

(d) Context switches

12. [1] The bootloader's primary role is to:

(a) Translate high-level code into assembly

(b) Load the operating system kernel into memory

(c) Optimize machine code for performance

(d) Start user applications

13. [1] Consider the following statements about *dlopen*(), *dlsym()*, and dlclose() when dynamically loading shared libraries in Linux. Which of the following is **true**?

(a) You must always call dlclose() immediately after *dlopen()* to avoid resource leaks, even if you plan to use symbols retrieved via *dlsym()* later.

(b) After calling *dlopen()* successfully, calling *dlsym()* on the returned handle retrieves the address of a symbol, but failure to call dlclose() will keep the library loaded indefinitely.

(c) Calling dlclose() invalidates all symbol pointers obtained from *dlsym()* for that library, so the pointers must not be used after dlclose().

(d) The order of calling *dlopen()*, *dlsym()*, and dlclose() is flexible, and the dynamic linker automatically manages undefined behavior from incorrect usage.

14. [1] At runtime, the stack is mainly used to store:

(a) Machine code

(b) Global variables

(c) Function calls and local variables

(d) Dynamically allocated memory

15. [1] The 32-bit integer 0x12345678 is stored starting at address 0x1000. Which option shows the correct byte order in *little endian*?

(a) `0x1000: 12, 0x1001: 34, 0x1002: 56, 0x1003: 78`

(b) `0x1000: 78, 0x1001: 56, 0x1002: 34, 0x1003: 12`

(c) `0x1000: 78, 0x1001: 12, 0x1002: 34, 0x1003: 56`

(d) `0x1000: 34, 0x1001: 12, 0x1002: 78, 0x1003: 56`

16. [1] Out-of-order execution mainly addresses:

(a) Branch prediction failures

(b) Instruction-level parallelism under data dependencies

(c) Increasing cache sizes

(d) Disk I/O delays

17. [1] Which of the following is **not** a pipeline hazard?

(a) Structural

(b) Data

(c) Control

(d) Algorithmic

18. [1] Which memory is implemented using SRAM cells?

(a) Main memory

(b) Cache

(c) Disk

(d) ROM

19. [1] In the 5-stage pipeline (IF, ID, EX, MEM, WB), a RAW (read-after-write) conflict is an example of:

(a) Structural hazard

(b) Data hazard

(c) Control hazard

(d) Memory hazard

20. [1]Which of the following statements about MMX and SSE instruction sets is correct?

(a) MMX instructions operate on 128-bit registers and support floating-point SIMD operations.

(b) SSE2 extends MMX by adding support for double-precision floating-point SIMD instructions and wider registers compared to MMX.

(c) SSE3 is an older technology than MMX and only supports integer SIMD operations.

(d) MMX registers and SSE registers can be used interchangeably without saving/loading changes due to being the same physical resource.

21. [1] Which of the following statements is correct regarding the order of libraries specified on the command line when linking a program with dependent libraries?

(a) The linker searches for undefined symbols in all listed libraries simultaneously, so the order of libraries does not affect linking outcomes.

(b) The linker processes libraries from left to right; if one library depends on symbols defined in another, the dependent library should come first and the library providing the definitions should come after it.

(c) The linker always processes libraries from right to left, choosing the last listed library to resolve dependencies.

(d) Library linking order only matters for dynamic libraries, not for static ones.

22. [1] SIMD is most effective for:

(a) Independent web requests

(b) Matrix multiplication

(c) File I/O

(d) Interrupt handling

23. [1] Which instruction is commonly used on x86-64 to invoke a modern system call?

(a) int 0x80

(b) syscall

(c) trap 0x1

(d) svc 0

24. [1] Which of the following is a role of the loader?

(a) Translating source code to assembly

(b) Resolving symbol addresses

(c) Placing the program in memory and starting execution

(d) Generating object files

25. [1] A "watchpoint" in a debugger is typically used to:

(a) Pause program at a specific code line always.

(b) Break when a variable's value changes (memory location watched).

(c) Track function call counts only.

(d) Only log program output to a file.

Section 2. **Fill in the blank**

26. [1] The segment that contains uninitialized global variables is called the Data segment.

27. [1] The system call interface acts as a bridge between user space and kernel space.

28. [1] The L1 cache is typically located closer to the processor core.

29. [1] GPUs are particularly well suited for _____ workloads.

30. [1] In pipelining, when a later instruction depends on the result of an earlier instruction, it is called a Data hazard.

31. [1] The compiler phase that checks types and declarations is called Symantic Analyser

32. [2] The name of the object file where the address of _start of every C programs *startup code* is _____

33. [1] If a superscalar processor can fetch and issue 2 instructions per cycle but the instruction stream has many dependencies, the effective IPC (instructions per cycle) will be close to . 1

34. [1] Fill in the blank: In a process memory layout, the segment that typically contains uninitialized global variables is called the Data segment.

Section 3. **Short Answer**

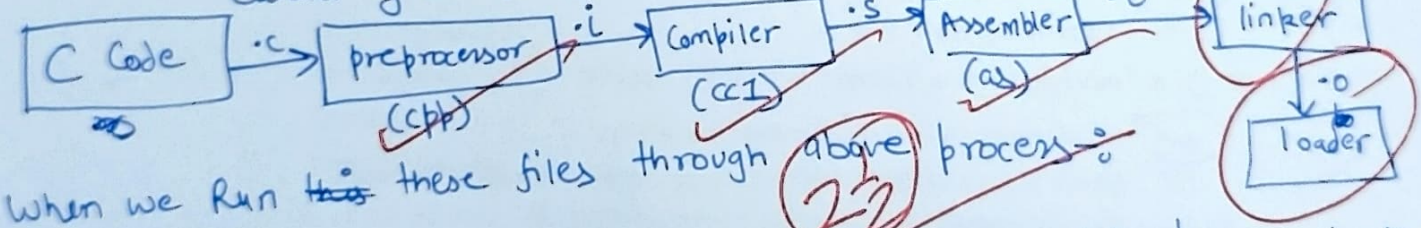
35. [3] You have two C source files:

```
// file1.c
int f1() { return 42; }

// file2.c
#include <stdio.h>
extern int f1();
int main() { printf("%d\n", f1()); }
```

Explain, in detailed steps, how these files become an executable on a Unix system. Mention preprocessing, compilation, assembly, object files, linking (symbol resolution, relocation), and what the loader does at program start.

Currently both files are in .c format



When we Run ~~this~~ these files through above process :-

1) preprocessor will replace the code of #include<stdio.h> by copying the relevent code from header file in file2. It will also convert both .c files to .i.

2) Compiler will check for compile time errors and convert the format of files from .i to .s (for both files). Compiler will also perform lexe generation, symbol table creation, code optimization, syntax test, etc. Denoted by

3) Assembler It's responsibility is to convert .s files to ~~.ot~~ .o files making code files binary. Its represented by as or gas. These are relocatable object files

4) Linker - It takes relocatable object files and creates executable object file (also in .o but not same). Represented by ld

5) Loader finally Loads and starts execution of program files in memory, it ready to run on ~~s~~ System.

Set D

36. [3] Why does every C program need a routine/function called main?

C programming is defined in a way such that main is always the first function which will get executed no matter where its written in code, it is done to ensure out of order function declaration in code does not lead to out of order execution.

1

37. [4] GDB uses the software breakpoint mechanism to pause execution of a debugged program.

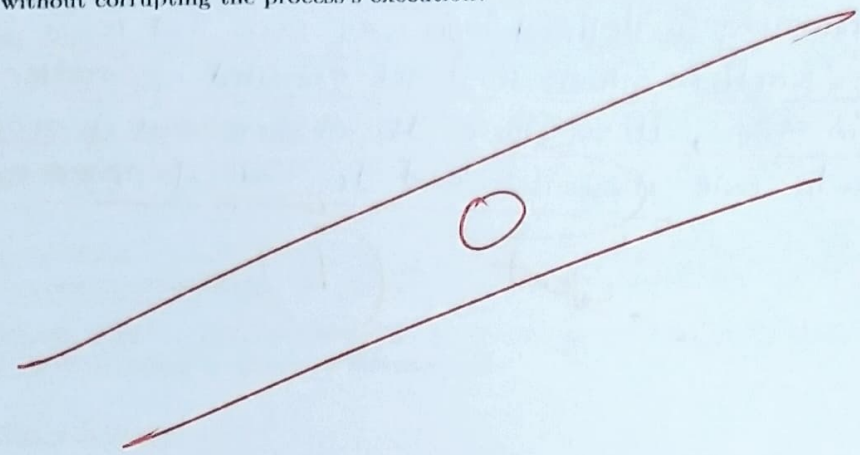
- Explain how GDB internally sets and manages breakpoints, including what changes occur in the program's memory and how GDB restores execution after the breakpoint is hit.

In the code we can set the breakpoints, when breakpoints occure, system calls code of breakpoint and showcases memory and instruction log till that point in terminal, its like a function call, it takes the control and displays data till control is not given back by user, once user allows it it returns control back to user

- When attaching to a running process (e.g., via gdb -p <pid>), how does GDB gain control of the target process, discover its current state, and safely establish breakpoints

10.

without corrupting the process's execution?



38. [3] (a) Explain how a C library function like **printf** eventually causes data to be written to the terminal (cover user-space buffering, glibc, write syscall). (b) Outline the high-level steps required to add a simple new system call to the Linux kernel (editing syscall table, implementing kernel handler, rebuilding kernel).

(a) (i) When a code is written, the printf has its definition in include <stdio.h> from there it gets now how to access the code from n as printf's code to perform write ritten in system.
urrently program was in user space so eing its definition, to perform system t needs to shift to kernel mode ed on type of linking done, its code glibc.a (static linking) or glibc.so (Dynamic linking)

(b) steps to add System call
(i) Define kernel code for new system call
(ii) write a make file for new system call
(iii) Integrate new system call to already existing makefile of kernel
(iv) Recompile the kernel code
(v) Restart the kernel / system terminal

(IV) After identifying type of linking and accessing code, it performs syste call (Syscall) in kernel mode.

39. [3] Write a small C function [*void show_bytes(int x)*]to print the byte representations of sample data objects

```
#include <stdio.h>
int main()
{ void show_bytes(int n)
{ int n= x; long val=0; int i=1;
while(n>0)
{
  int t= n%2;
  val = val + t*i;
   i = i*10;
  n=n/2;
}
printf("%ld", val);
}
```

10001 → 17
1000
5
5%2 → 1
2%2 → 1
1%2 → 1
10